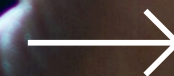


MLUP: Statistical Learning Theory

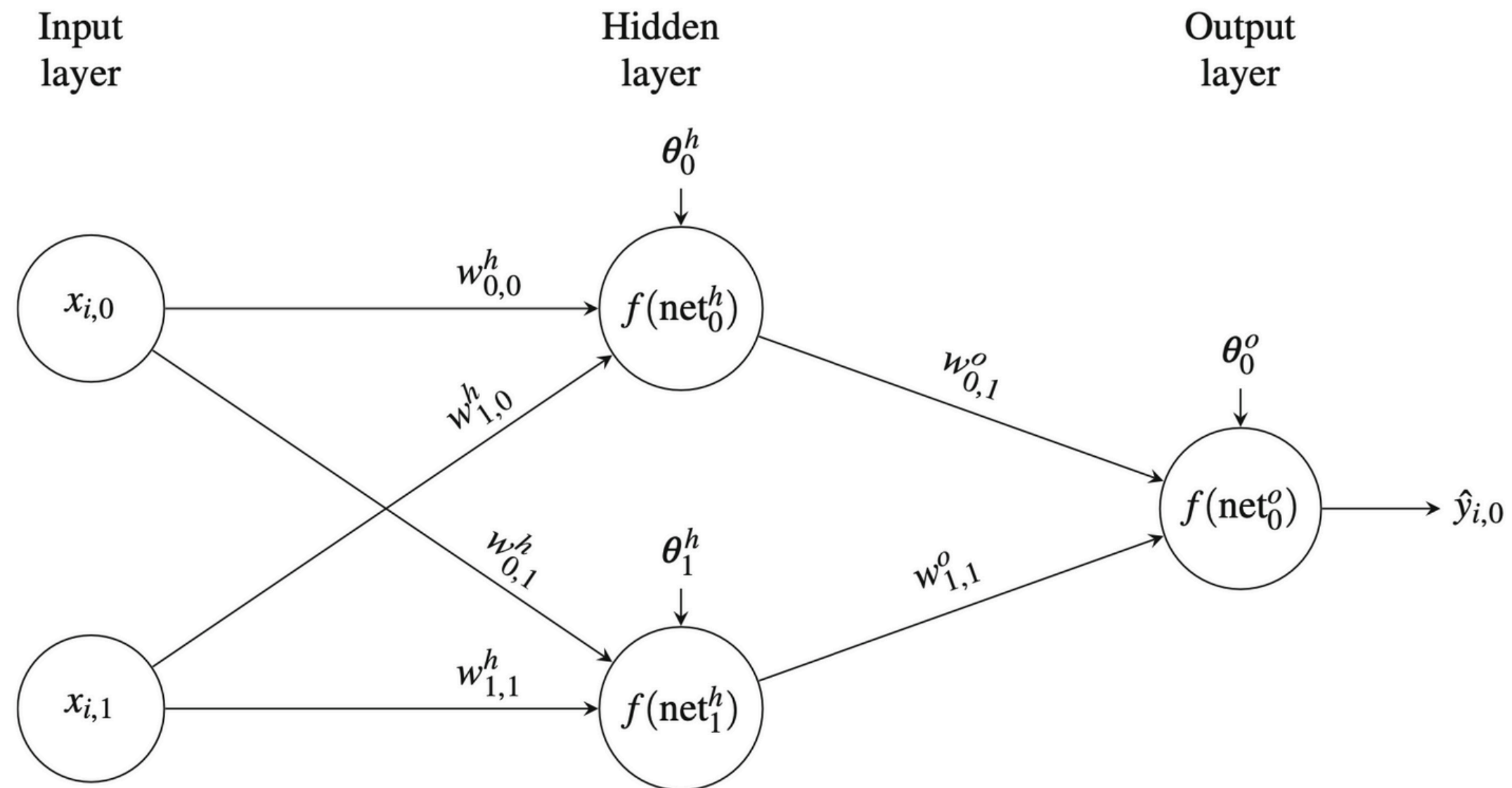
Presented by Gabriel Rodrigues Palma



Machine Learning using
Python (MLUP01)



Multilayer Perceptron II



Multilayer Perceptron II

We formalize MLP with the squared-error function:

$$E_i^2 = (y_i - f(\text{net}_i))^2, \quad (1.19)$$

in which $f(\text{net}_i)$ corresponds to the output MLP produces for example i . Rewriting term $f(\text{net}_i)$:



Multilayer Perceptron II

We formalize MLP with the squared-error function:

$$E_i^2 = (y_i - f(\text{net}_i))^2, \quad (1.19)$$

in which $f(\text{net}_i)$ corresponds to the output MLP produces for example i . Rewriting term $f(\text{net}_i)$:

$$E_i^2 = \sum_k (y_{i,k} - f_k^o(\text{net}_{i,k}^o))^2$$

Multilayer Perceptron II

We formalize MLP with the squared-error function:

$$E_i^2 = (y_i - f(\text{net}_i))^2, \quad (1.19)$$

in which $f(\text{net}_i)$ corresponds to the output MLP produces for example i . Rewriting term $f(\text{net}_i)$:

$$E_i^2 = \sum_k (y_{i,k} - f_k^o(\text{net}_{i,k}^o))^2 = \left(y_{i,k} - f_k^o \left[\sum_j f_j^h(\text{net}_j^h) w_{k,j}^o + \theta_k^o \right] \right)^2, \quad (1.20)$$

Multilayer Perceptron II

1. Equation (1.21) to adapt every weight $w_{j,l}^h$ connecting input neuron l to hidden neuron j ;
2. Equation (1.22) to adapt every weight $w_{k,j}^o$ connecting hidden neuron j to output neuron k .

$$w_{j,l}^h(t+1) = w_{j,l}^h(t) - \eta \frac{\partial E_i^2}{\partial w_{j,l}^h} \quad (1.21)$$

$$w_{k,j}^o(t+1) = w_{k,j}^o(t) - \eta \frac{\partial E_i^2}{\partial w_{k,j}^o} \quad (1.22)$$

Multilayer Perceptron II

To complement, Eqs. (1.23) and (1.24) provide the GD method to adapt the free variable θ for every neuron at either the hidden or the output layer along iterations:

$$\theta_j^h(t + 1) = \theta_j^h(t) - \eta \frac{\partial E_i^2}{\partial \theta_j^h}, \quad (1.23)$$

$$\theta_k^o(t + 1) = \theta_k^o(t) - \eta \frac{\partial E_i^2}{\partial \theta_k^o}. \quad (1.24)$$

Multilayer Perceptron II

Despite the Gradient Descent equations are the same for weights in both layers as well as for thetas, the partial derivatives:

$$\frac{\partial E_i^2}{\partial w_{j,l}^h}, \frac{\partial E_i^2}{\partial w_{k,j}^o}, \frac{\partial E_i^2}{\partial \theta_j^h}, \text{ and } \frac{\partial E_i^2}{\partial \theta_k^o}$$

change. Consequently, we must compute the partial derivatives for this problem XOR to exemplify the MLP formulation, however this solution still supports a general-purpose algorithm.

Multilayer Perceptron II

The partial derivatives for the output layer are found in advance, once they are simpler to obtain:

$$\frac{\partial E_i^2}{\partial w_{k,j}^o} = \frac{\partial \sum_k (y_{i,k} - \hat{y}_{i,k})^2}{\partial w_{k,j}^o}$$



Multilayer Perceptron II

The partial derivatives for the output layer are found in advance, once they are simpler to obtain:

$$\begin{aligned}\frac{\partial E_i^2}{\partial w_{k,j}^o} &= \frac{\partial \sum_k (y_{i,k} - \hat{y}_{i,k})^2}{\partial w_{k,j}^o} \\ &= \frac{\sum_k \left(y_{i,k} - f_k^o \left(\sum_j f_j^h(\text{net}_j^h) w_{k,j}^o + \theta_k^o \right) \right)^2}{\partial w_{k,j}^o},\end{aligned}\tag{1.25}$$

Multilayer Perceptron II

The differentiation employs the chain rule as follows:

$$\frac{\partial E_i^2}{\partial w_{k,j}^o} = \sum_k 2(y_{i,k} - \hat{y}_{i,k}) \frac{\partial (y_{i,k} - \hat{y}_{i,k})}{\partial w_{k,j}^o},$$

as $y_{i,k}$ is a constant defining the expected class to be produced by output neuron k , assuming zero while deriving in terms of $w_{k,j}^o$:

$$\frac{\partial E_i^2}{\partial w_{k,j}^o} = \sum_k 2(y_{i,k} - \hat{y}_{i,k}) \frac{\partial -\hat{y}_{i,k}}{\partial w_{k,j}^o},$$

Multilayer Perceptron II

$$\frac{\partial E_i^2}{\partial w_{k,j}^o} = \sum_k 2(y_{i,k} - \hat{y}_{i,k}) \frac{\partial - \hat{y}_{i,k}}{\partial w_{k,j}^o},$$

thus, we still need to solve the derivative $\frac{\partial - \hat{y}_{i,k}}{\partial w_{k,j}^o}$. As the reader may recall, $\hat{y}_{i,k} = f_k^o(\text{net}_{i,k}^o)$, i.e., the output value produced by output neuron k is the result of the sigmoid function, defined in Eq. (1.18):



Multilayer Perceptron II

$$\frac{\partial - \hat{y}_{i,k}}{\partial w_{k,j}^o} = \frac{\partial - f_k^o(\text{net}_{i,k}^o)}{\partial w_{k,j}^o}$$

Multilayer Perceptron II

$$\begin{aligned}\frac{\partial - \hat{y}_{i,k}}{\partial w_{k,j}^o} &= \frac{\partial - f_k^o(\text{net}_{i,k}^o)}{\partial w_{k,j}^o} \\ &= \frac{\partial - (1 + e^{-\text{net}_{i,k}^o})^{-1}}{\partial w_{k,j}^o}\end{aligned}$$

Multilayer Perceptron II

$$\begin{aligned}\frac{\partial - \hat{y}_{i,k}}{\partial w_{k,j}^o} &= \frac{\partial - f_k^o(\text{net}_{i,k}^o)}{\partial w_{k,j}^o} \\ &= \frac{\partial - (1 + e^{-\text{net}_{i,k}^o})^{-1}}{\partial w_{k,j}^o} \\ &= \frac{\partial - \left(1 + e^{-\left[\sum_j f_j^h(\text{net}_j^h)w_{k,j}^o + \theta_k^o\right]}\right)^{-1}}{\partial w_{k,j}^o}.\end{aligned}$$

Multilayer Perceptron II

We know the sigmoid function (Eq. (1.18)) has the following derivative in terms of net:

$$\frac{\partial f(\text{net})}{\partial \text{net}} = f(\text{net})(1 - f(\text{net})),$$



Multilayer Perceptron II

We know the sigmoid function (Eq. (1.18)) has the following derivative in terms of net:

$$\frac{\partial f(\text{net})}{\partial \text{net}} = f(\text{net})(1 - f(\text{net})),$$

what simplifies our formulation:



Multilayer Perceptron II

We know the sigmoid function (Eq. (1.18)) has the following derivative in terms of net:

$$\frac{\partial f(\text{net})}{\partial \text{net}} = f(\text{net})(1 - f(\text{net})),$$

what simplifies our formulation:

$$\frac{\partial -f_k^o(\text{net}_{i,k}^o)}{\partial w_{k,j}^o} = - \left[f_k^o(\text{net}_{i,k}^o)(1 - f_k^o(\text{net}_{i,k}^o)) \frac{\partial \text{net}_{i,k}^o}{\partial w_{k,j}^o} \right],$$

Multilayer Perceptron II

what simplifies our formulation:

$$\frac{\partial -f_k^o(\text{net}_{i,k}^o)}{\partial w_{k,j}^o} = - \left[f_k^o(\text{net}_{i,k}^o)(1 - f_k^o(\text{net}_{i,k}^o)) \frac{\partial \text{net}_{i,k}^o}{\partial w_{k,j}^o} \right],$$

so, we still need to find the following partial derivative:

$$\frac{\partial \text{net}_{i,k}^o}{\partial w_{k,j}^o} = \frac{\partial \sum_j f_j^h(\text{net}_{i,j}^h) w_{k,j}^o + \theta_k^o}{\partial w_{k,j}^o} = f_j^h(\text{net}_{i,j}^h).$$

Multilayer Perceptron II

Connecting all terms, the update rule for weights at the output layer is:

$$w_{k,j}^o(t+1) = w_{k,j}^o(t) - \eta \frac{\partial E_i^2}{\partial w_{k,j}^o}$$

$$w_{k,j}^o(t+1) = w_{k,j}^o(t) - \eta 2(y_i - \hat{y}_{i,k}) \frac{\partial - \left[f_k^o(\text{net}_{i,k}^o) \right]}{\partial w_{k,j}^o}$$

Multilayer Perceptron II

$$w_{k,j}^o(t+1) = w_{k,j}^o(t)$$

$$- \eta 2(y_i - f_k^o(\text{net}_{i,k}^o)) \left(- \left[f_k^o(\text{net}_{i,k}^o)(1 - f_k^o(\text{net}_{i,k}^o)) \right] \right) \frac{\partial \text{net}_{i,k}^o}{\partial w_{k,j}^o}$$

$$w_{k,j}^o(t+1) = w_{k,j}^o(t)$$

$$- \eta 2(y_i - f_k^o(\text{net}_{i,k}^o)) \left(- \left[f_k^o(\text{net}_{i,k}^o)(1 - f_k^o(\text{net}_{i,k}^o)) \right] \right) f_j^h(\text{net}_{i,j}^h).$$

Multilayer Perceptron II

Similarly, the Gradient Descent formulated for theta from output neuron k is:

$$\theta_k^o(t+1) = \theta_k^o(t) - \eta 2(y_i - f_k^o(\text{net}_{i,k}^o)) \left(- \left[f_k^o(\text{net}_{i,k}^o)(1 - f_k^o(\text{net}_{i,k}^o)) \right] \right) 1,$$



Multilayer Perceptron II

Similarly, the Gradient Descent formulated for theta from output neuron k is:

$$\theta_k^o(t+1) = \theta_k^o(t) - \eta 2(y_i - f_k^o(\text{net}_{i,k}^o)) (-[f_k^o(\text{net}_{i,k}^o)(1 - f_k^o(\text{net}_{i,k}^o))]) 1,$$

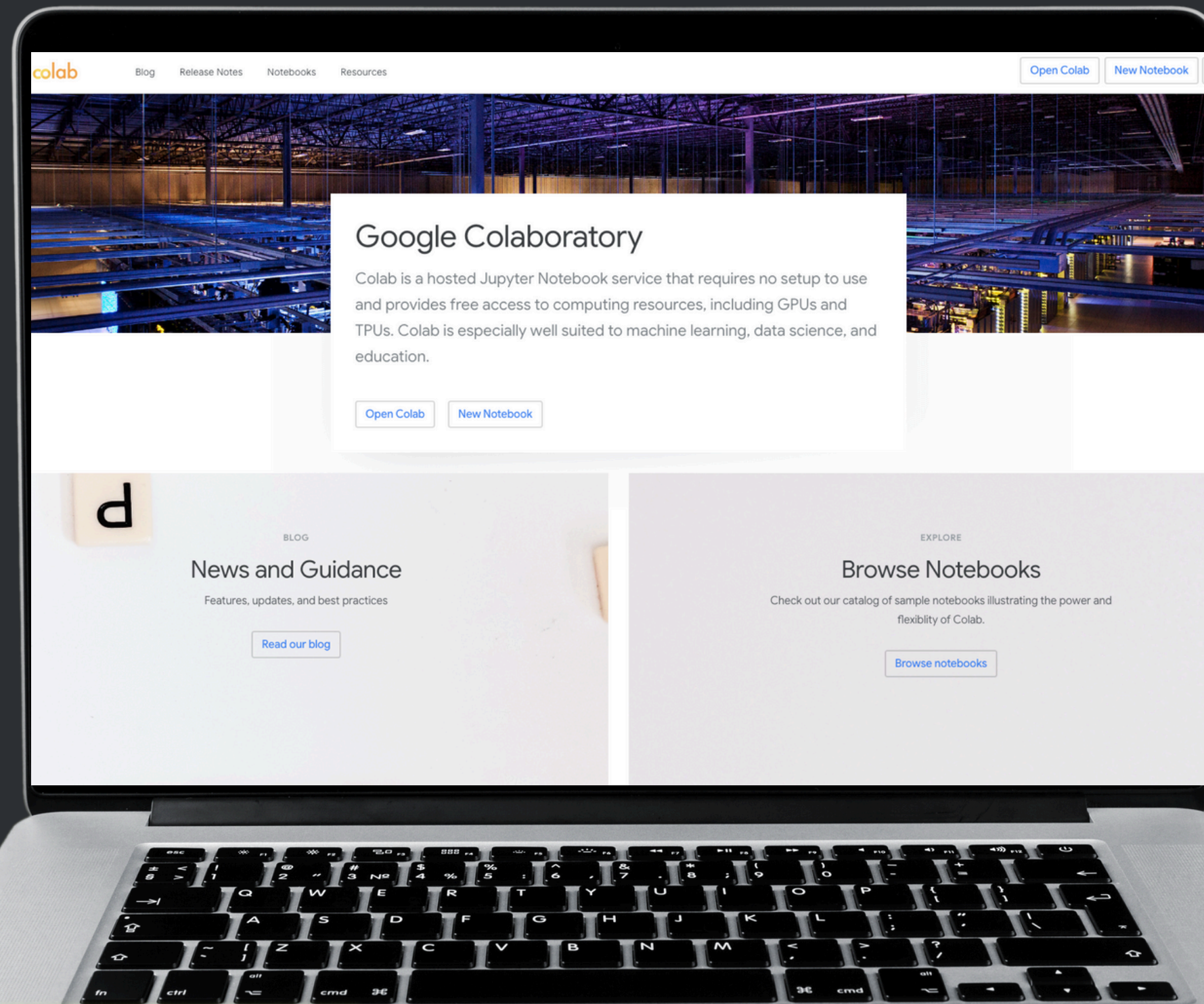
which has another term instead of $f_j^h(\text{net}_{i,j}^h)$, once the derivative:

$$\frac{\partial \text{net}_{i,k}^o}{\partial \theta_k^o} = \frac{\partial [\sum_j f_j^h(\text{net}_{i,j}^h) w_{k,j}^o + \theta_k^o]}{\partial \theta_k^o} = 1,$$

results in the number 1. Finally, we have the update rules for weights and thetas at the output layer.

Multilayer Perceptron II

Hands-on activity:



Machine Learning
using Python (MLUP01)

